

Gabarito — Lista de exercícios — Árvore Binária de Busca

1. Alternativa correta: (c).

2. Alternativa correta: (a).

3. (a) Nós folha: D, J, P, T.

(b) Grau da árvore: 2.

(c) Altura da árvore: 2 arestas (ou 3 níveis, dependendo da convenção adotada).

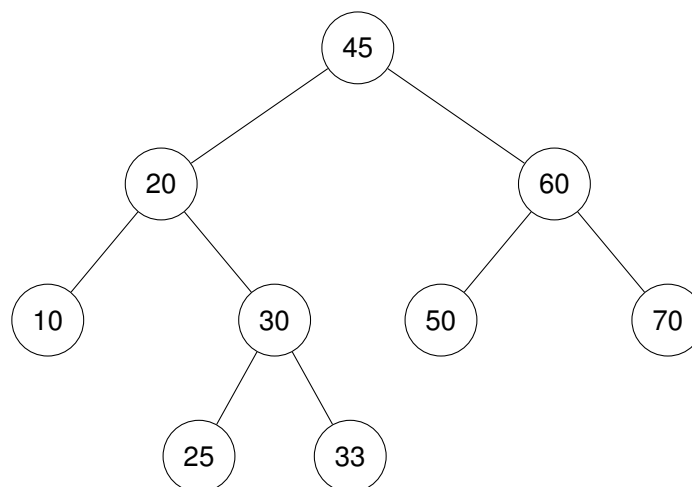
(d) Descendentes de H: D e J.

4. (a) Pré-ordem: 35, 18, 11, 24, 52, 41, 67

(b) Em ordem: 11, 18, 24, 35, 41, 52, 67

(c) Pós-ordem: 11, 24, 18, 41, 67, 52, 35

5. Árvore resultante:



6. Fila resultante após a travessia em ordem:

9, 18, 21, 24, 32, 35, 39, 48, 56

7. **Alternativa correta:** (c) b – e **Errata:** na questão original, a alternativa (c) deve ser lida como b – e, e não como b – d.

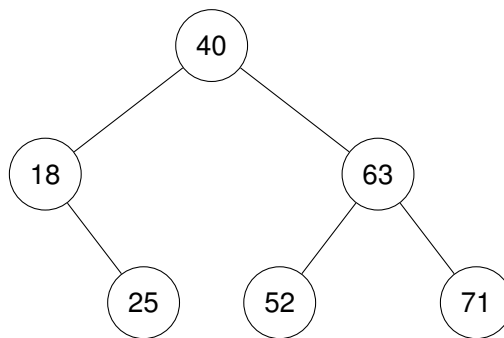
8. I. V

II. V

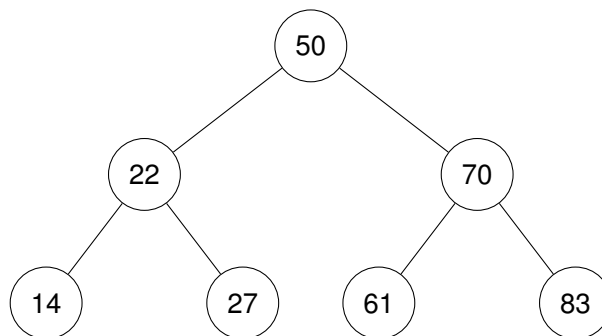
III. F

IV. V

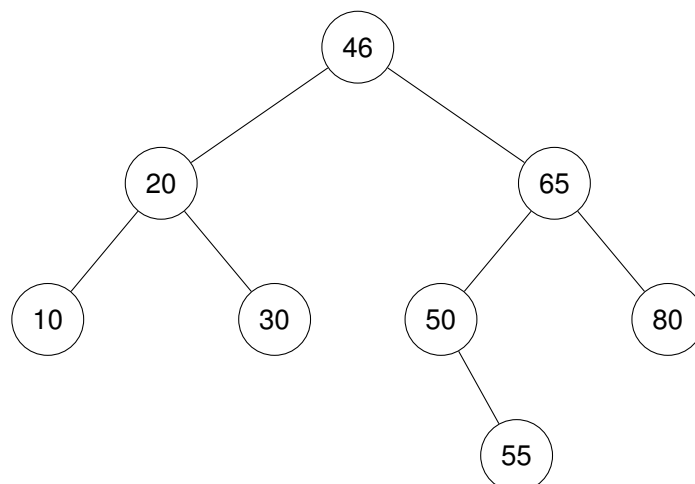
9. Árvore após remover 11:



10. Árvore após remover 31:



11. Árvore após remover 40 usando o sucessor em-order:



12. • I. V
• II. V
• III. V
• IV. F
• V. V

```
13. NoArv* buscarABB(NoArv* raiz, int valor) {
    if (raiz == NULL || raiz->info == valor)
        return raiz;

    if (valor < raiz->info)
        return buscarABB(raiz->esq, valor);
    else
        return buscarABB(raiz->dir, valor);
}
```

```
14. int contarNos(NoArv* raiz) {
    if (raiz == NULL)
        return 0;

    return 1 + contarNos(raiz->esq) + contarNos(raiz->dir);
}
```

```
15. int altura(NoArv* raiz) {
    int he, hd;

    if (raiz == NULL)
        return -1;

    he = altura(raiz->esq);
    hd = altura(raiz->dir);

    return 1 + (he > hd ? he : hd);
}
```

```
16. int contarFolhas(NoArv* raiz) {
    if (raiz == NULL)
        return 0;

    if (raiz->esq == NULL && raiz->dir == NULL)
        return 1;

    return contarFolhas(raiz->esq) + contarFolhas(raiz->dir);
}
```

```
17. void mostrarEmOrdem(NoArv* raiz) {
    if (raiz != NULL) {
        mostrarEmOrdem(raiz->esq);
        printf("%d ", raiz->info);
        mostrarEmOrdem(raiz->dir);
    }
}
```

```
18. int menorABB(NoArv* raiz) {
    while (raiz->esq != NULL)
        raiz = raiz->esq;
}
```

```

    return raiz->info;
}

```

```

19. void espelhaArvore(NoArv* raiz) {
    NoArv* temp;

    if (raiz == NULL)
        return;

    temp = raiz->esq;
    raiz->esq = raiz->dir;
    raiz->dir = temp;

    espelhaArvore(raiz->esq);
    espelhaArvore(raiz->dir);
}

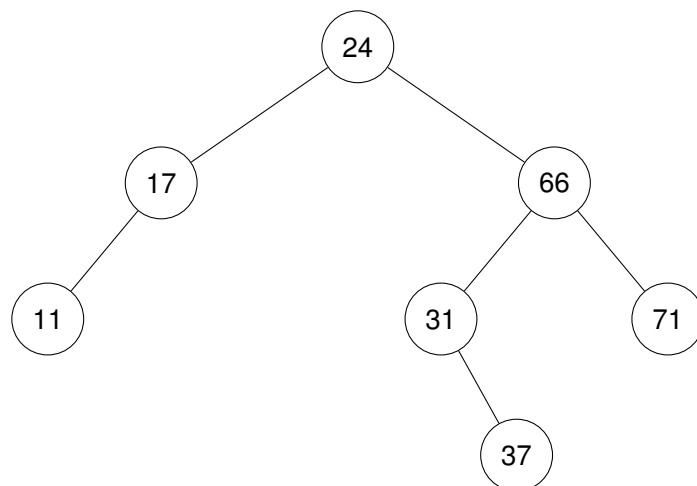
```

20. **Árvore A:** completa; cheia; balanceada; grau 2; ABB válida.

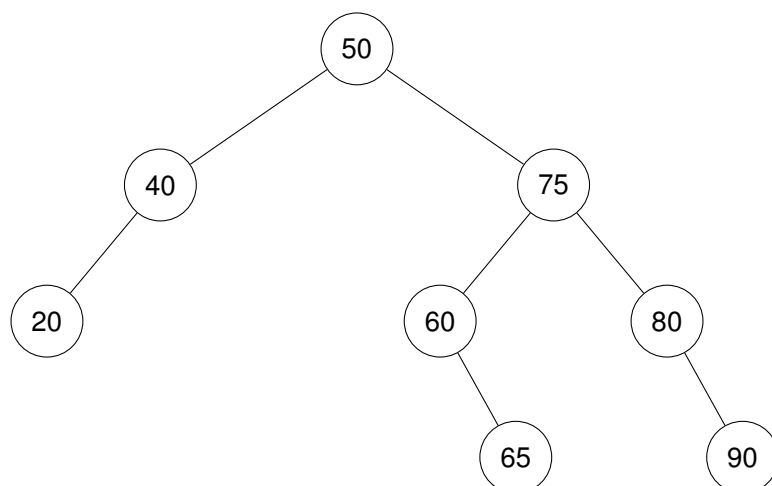
Árvore B: incompleta; não cheia; não balanceada; grau 2; ABB inválida. É inválida porque o nó 30 está na subárvore esquerda de 25, mas é maior que 25.

Árvore C: incompleta; não cheia; balanceada; grau 2; ABB válida.

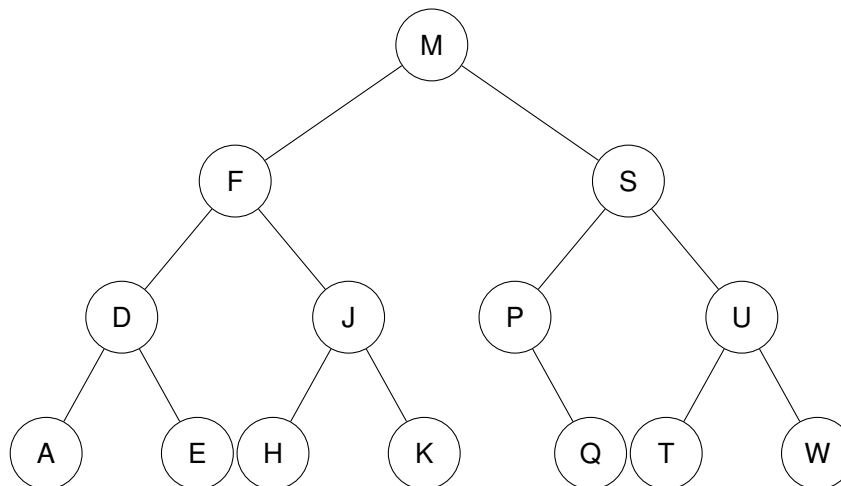
21. Árvore final:



22. Árvore final:



23. Árvore resultante:



24. (a) Raiz: 48

(b) Pai de 58: 55

(c) O nó 37 será filho **direito** de 26

(d) Quantidade de nós folha: 5

25. Alternativa correta: (c).

26. Porque, em uma ABB, todos os valores da subárvore esquerda são menores que a raiz e todos os valores da subárvore direita são maiores que a raiz. Como a travessia em ordem visita **esquerda – raiz – direita**, os valores são produzidos em ordem crescente.

```
27. int contarPares(NoArv* raiz) {  
    if (raiz == NULL)  
        return 0;  
  
    return (raiz->info % 2 == 0)  
        + contarPares(raiz->esq)  
        + contarPares(raiz->dir);  
}
```

```
28. int abbIguais(NoArv* a1, NoArv* a2) {  
    if (a1 == NULL && a2 == NULL)  
        return 1;  
  
    if (a1 == NULL || a2 == NULL)  
        return 0;  
  
    return (a1->info == a2->info)  
        && abbIguais(a1->esq, a2->esq)  
        && abbIguais(a1->dir, a2->dir);  
}
```

```
29. int ehCheia(NoArv* raiz) {  
    if (raiz == NULL)  
        return 1;  
}
```

```

        if ((raiz->esq == NULL && raiz->dir != NULL) ||
            (raiz->esq != NULL && raiz->dir == NULL))
            return 0;

        return ehCheia(raiz->esq) && ehCheia(raiz->dir);
    }

```

30. int mostraCaminho(NoArv* raiz, int valor) {
 NoArv* p;

```

    if (buscarABB(raiz, valor) == NULL)
        return 0;

    p = raiz;
    while (p != NULL) {
        printf("%d ", p->info);

        if (valor == p->info)
            return 1;
        else if (valor < p->info)
            p = p->esq;
        else
            p = p->dir;
    }

    return 0;
}

```